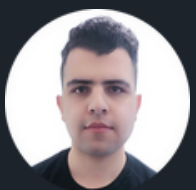


Async I/O Calls

Fetching data asynchronously from an external API is a common async scenario.

```
using System;
using System.Net.Http;
using System.Threading.Tasks;
class Program
{
    static async Task Main()
    {
        string apiUrl = "https://jsonplaceholder.typicode.com/posts/1";
        string result = await GetApiResponseAsync(apiUrl);

        Console.WriteLine("API Response:");
        Console.WriteLine(result);
    }
    static async Task<string> GetApiResponseAsync(string apiUrl)
    {
        using HttpClient client = new HttpClient();
        return await client.GetStringAsync(apiUrl);
    }
}
```



Elliot One

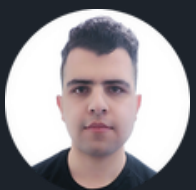
Avoid Using Task.Result

Using Task.Result on the **main thread** can lead to a **deadlock** scenario in asynchronous code.

```
using System;
using System.Threading.Tasks;
class Program
{
    static async Task Main()
    {
        Task<int> asyncOperation = PerformAsyncOperation();

        // Incorrectly using Task.Result on the main thread
        int result = asyncOperation.Result;

        Console.WriteLine($"Result: {asyncOperation.Result}");
    }
    static async Task<int> PerformAsyncOperation()
    {
        // Simulating an asynchronous operation
        await Task.Delay(1000);
        return 42;
    }
}
```



Elliot One

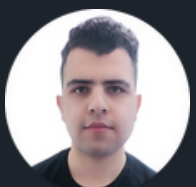
Avoid Using Task.Wait()

Incorrect usage of Task.Wait() on the **main thread** can cause a **deadlock**.

```
using System;
using System.Threading.Tasks;
class Program
{
    static void Main()
    {
        Task<int> asyncOperation = PerformAsyncOperation();

        // Incorrectly using Task.Wait on the main thread
        asyncOperation.Wait();

        Console.WriteLine($"Result: {asyncOperation.Result}");
    }
    static async Task<int> PerformAsyncOperation()
    {
        // Simulating an asynchronous operation
        await Task.Delay(1000);
        return 42;
    }
}
```



Elliot One

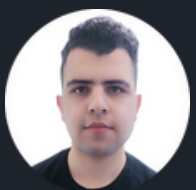
Using **await**

This example illustrates the **correct way** to **wait for the completion** of an asynchronous task using the **await** keyword.

```
using System;
using System.Threading.Tasks;
class Program
{
    static async Task Main()
    {
        Task<int> asyncOperation = PerformAsyncOperation();

        // Properly using await to wait for the completion
        int result = await asyncOperation;

        Console.WriteLine($"Result: {result}");
    }
    static async Task<int> PerformAsyncOperation()
    {
        // Simulating an asynchronous operation
        await Task.Delay(1000);
        return 42;
    }
}
```

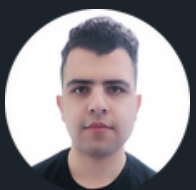


Elliot One

Offloading a CPU-bound operation

In the next example, `Task.Run()` offloads CPU-bound operation (calculating square roots) to a separate thread.

This prevents the CPU-bound work from blocking the main thread, ensuring responsiveness.



Elliot One

Offloading a CPU-bound operation

```
using System;
using System.Threading.Tasks;
class Program
{
    static async Task Main()
    {
        Console.WriteLine("Start of the program.");

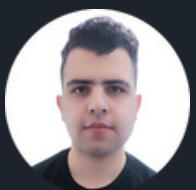
        // Example 1: Simulating an asynchronous operation
        await PerformAsyncOperation();

        // Example 2: Offloading CPU-bound work with Task.Run
        await Task.Run(() => OffloadCpuBoundWork());

        Console.WriteLine("End of the program.");
    }
    static async Task PerformAsyncOperation()
    {
        Console.WriteLine("Start of asynchronous operation.");
        await Task.Delay(2000);
        Console.WriteLine("End of asynchronous operation.");
    }
    static void OffloadCpuBoundWork()
    {
        Console.WriteLine("Start of CPU-bound work.");

        for (int i = 0; i < 5_000_000; i++)
        {
            var result = Math.Sqrt(i);
        }

        Console.WriteLine("End of CPU-bound work.");
    }
}
```



Elliot One

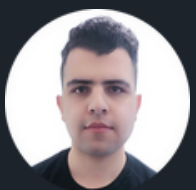
Using `WhenAll()`

`Task.WhenAll()` enables one to await **the completion of multiple** asynchronous operations.

```
public async Task ExecuteMultipleTasks()
{
    Task<int> task1 = PerformAsyncOperation(1);
    Task<int> task2 = PerformAsyncOperation(2);
    Task<int> task3 = PerformAsyncOperation(3);

    // Task. WhenAll waits for all tasks to complete
    int[] results = await Task.WhenAll(task1, task2, task3);

    Console.WriteLine("Results:");
    foreach (int result in results)
    {
        Console.WriteLine(result);
    }
}
```



Elliot One

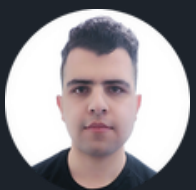
Using `WhenAny()`

`Task.WhenAny()` enables one to await the completion of **the swiftest asynchronous operation** of multiple operations.

```
public async Task AwaitFirstCompletedTask()
{
    Task<int> task1 = PerformAsyncOperation(1);
    Task<int> task2 = PerformAsyncOperation(2);
    Task<int> task3 = PerformAsyncOperation(3);

    // Task. WhenAny waits for the first task to complete
    Task<int> firstCompletedTask =
        await Task.WhenAny(task1, task2, task3);

    Console.WriteLine(
        $"The first task completed with result: {firstCompletedTask.Result}");
}
```



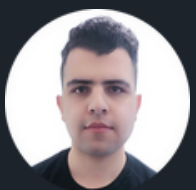
Elliot One

Using `WhenEach()`

`Task.WhenEach()` enables one to await multiple asynchronous operations and process their results in the order they complete.

```
public async Task AwaitTasksAsTheyComplete()
{
    Task<int> task1 = PerformAsyncOperation(1);
    Task<int> task2 = PerformAsyncOperation(2);
    Task<int> task3 = PerformAsyncOperation(3);

    // Task.WhenEach yields tasks in the order they complete
    await foreach (
        var completedTask in Task.WhenEach(task1, task2, task3))
    {
        int result = await completedTask;
        Console.WriteLine($"A task completed with result: {result}");
    }
}
```



Elliot One

Utilizing Cancellation Token

A cancellation token enables the **graceful interruption** of an asynchronous operation.

```
class Program
{
    static async Task Main()
    {
        Console.WriteLine("Start of the program.");

        // Create a cancellation token source
        CancellationTokensource cancellationTokensource =
            new CancellationTokensource();
        CancellationToken cancellationToken = cancellationTokensource.Token;

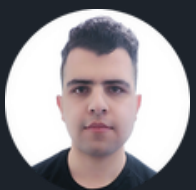
        // Cancel the operation after a delay (simulate user cancellation, etc.)
        cancellationTokensource.CancelAfter(2000);

        // Using a cancellation token in an asynchronous operation
        try
        {
            await PerformAsyncOperationWithCancellation(cancellationToken);
            Console.WriteLine("Async operation completed successfully.");
        }
        catch (OperationCanceledException ex)
        {
            Console.WriteLine($"Async operation was canceled: {ex.Message}");
            Console.WriteLine("End of the program.");
        }
    }

    static async Task PerformAsyncOperationWithCancellation(
        CancellationToken cancellationToken)
    {
        for (int i = 0; i < 10; i++)
        {
            // Simulating asynchronous work
            await Task.Delay(500);

            // Check for cancellation
            cancellationToken.ThrowIfCancellationRequested();

            Console.WriteLine($"Iteration {i + 1} completed.");
        }
    }
}
```



Elliot One

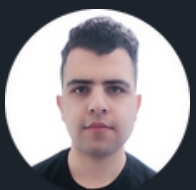
Exception Handling in Async Operations

Utilizing a **try-catch** block helps to handle exceptions raised by asynchronous tasks.

```
class Program
{
    static async Task Main()
    {
        Console.WriteLine("Start of the program.");

        // Handling exceptions in an asynchronous operation
        try
        {
            await PerformAsyncOperationWithException();
            Console.WriteLine("Async operation completed successfully.");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Async operation failed: {ex.Message}");
        }

        Console.WriteLine("End of the program.");
    }
    static async Task PerformAsyncOperationWithException()
    {
        // Simulating an asynchronous operation with an exception
        await Task.Delay(1000);
        throw new InvalidOperationException(
            "An intentional exception occurred.");
    }
}
```



Elliot One



Elliot One

Enjoyed Reading This?



Reshare and **Spread Knowledge.**